



Intro a la programación usando Anaconda

Python



Python
Guido Van Rossum
(Países Bajos, 1956)

Diciembre de 1989.
1991 primera versión pública 0.9.0.
1994 versión 1.0
2000 la versión 2.0
2008 versión 3.0





Lenguaje dinámicamente interpretado.

Tipado dinámico. No es necesario declarar los tipos de datos.

Sintaxis simple y económica.

Programa se lee de arriba hacia abajo.

Es orientado a objetos: Herencia múltiple, Ligadura dinamica. ☐

Polimorfismo.

Gratuito, y de libre distribución

Rápido aprendizaje.

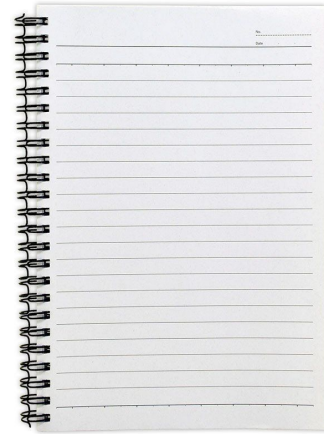
Debes ser ordenado.

■ .py

LENGUAJE - IDE



Lenguaje
Las palabras, reglas
sintácticas funciones y
métodos



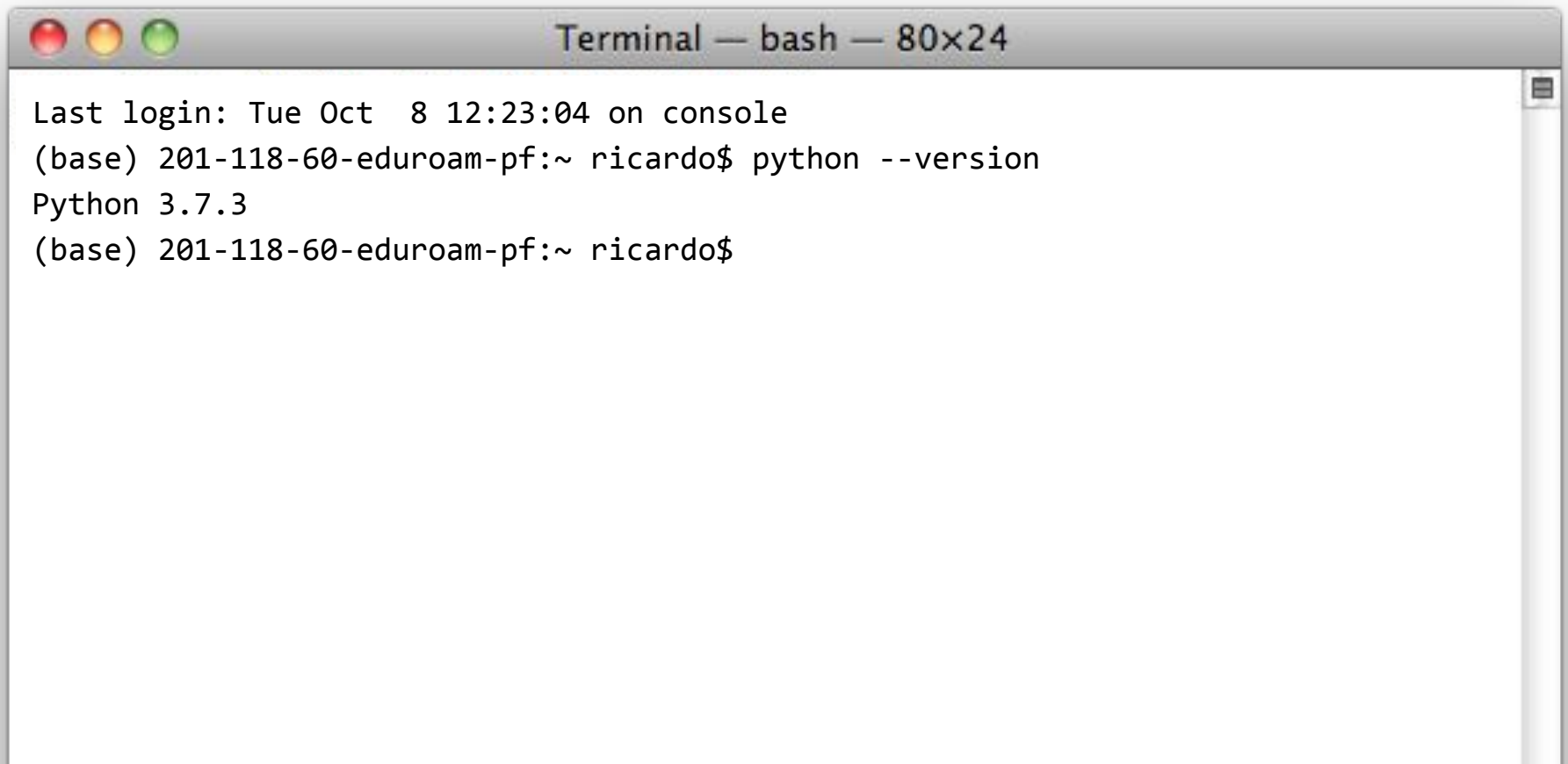
Editor
Para escribir y ejecutar
programas

QUÉ VERSIÓN DE PYTHON TENGO?

Ir a Terminal y escribir lo siguiente...

`python --version`

luego enter...



```
Terminal — bash — 80x24

Last login: Tue Oct  8 12:23:04 on console
(base) 201-118-60-eduroam-pf:~ ricardo$ python --version
Python 3.7.3
(base) 201-118-60-eduroam-pf:~ ricardo$
```

CONCEPTOS ELEMENTALES DE PROGRAMACIÓN

tipos de datos - valores se guardan de manera diferente en el computador

variables - crear espacios en la memoria para guardar datos

condicionales (if - else - elif) - hacer preguntas para controlar flujo del programa

ciclos (for - while) - repetir instrucciones *n* veces o según cierta condición

listas - guardar muchas variables

funciones - empaquetar instrucciones

clases - crear objetos complejos

TIPOS DE DATOS

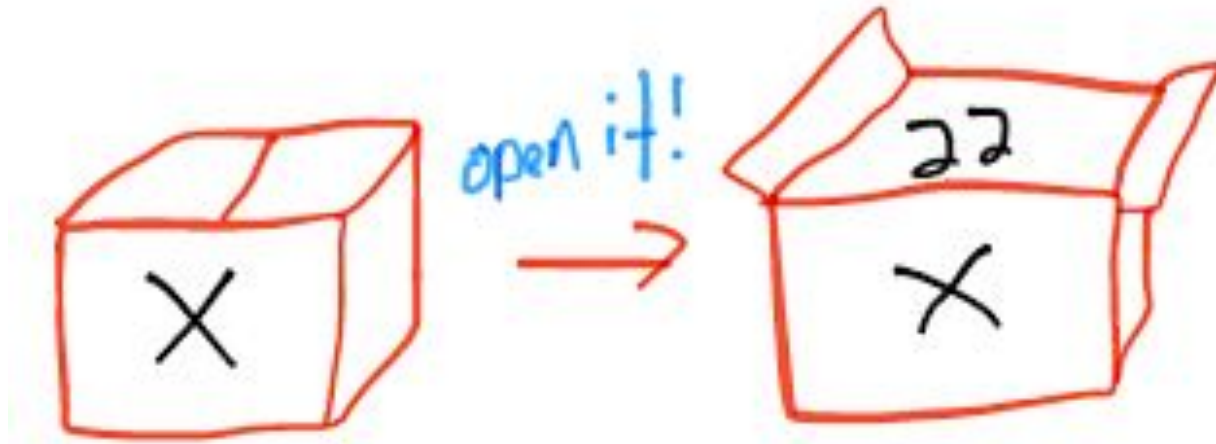
TIPOS DE DATOS

	<i>nombre</i>	<i>ejemplo</i>
Números	integer float complex	3 enteros 3.0 punto flotante 3 + 0j complejos
Texto	str	“Texto con comillas dobles” o ‘simples’
Booleano	bool	True, False
Type()	función para conocer tipo de dato	

VARIABLES

VARIABLES

Variables, una caja para guardar valores



a = 5

entero - int

b = 2 + 3

suma de enteros

dia = "martes"

String - cadena de caracteres - str

llueve = True

Booleano - bool

nombre

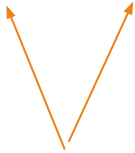
asignador

valor

Construir una variable

declarar y asignar `n = 5`

usar `print("tengo", n, "gatitos")`



se concatenan valores
separados por coma

concatenar es reunir,
no sumar...

Cuidado al usar signo “igual”

a = 5

asignación de valor a la variable
(en variables)

b == 2 + 3

comparación de dos valores
(en condicionales)

CONDICIONALES

if

else

elif

Instrucciones condicionales

Permiten ejecutar una o más instrucciones solamente si se cumple una condición

Condición

Expresión cuyo valor es de tipo bool. Dos valores

posibles: True ó False

True o False

Se puede pensar como una pregunta

Si pasa esto

has esto

Si llueve...

uso paraguas.

Si pasa lo otro

hacer otra cosa

Si no llueve...

no uso paraguas.

Es una pregunta

para comparar (evaluar) ciertos valores
y retorna la respuesta

$2 \leq 3?$

$3 \geq 2?$

$a == 5?$

$isOn == True?$

ojo:

= es asignar

== es comparar

Operadores Booleanos de comparación o relacionales retornan un valor de tipo bool.

Operador	Descripción
<code>a == b</code>	Retorna True ssi <i>a es igual a b</i>
<code>a != b</code>	Retorna True ssi <i>a es distinto de b</i>
<code>a < b</code>	Retorna True ssi <i>a es menor que b</i>
<code>a <= b</code>	Retorna True ssi <i>a es menor o igual que b</i>
<code>a > b</code>	Retorna True ssi <i>a es mayor que b</i>
<code>a >= b</code>	Retorna True ssi <i>a es mayor o igual que b</i>

Operadores de comparación o relacionales.

`==` (igualdad)
`!=` (desigualdad)
`<` (menor que)
`<=` (menor o igual que)
`>` (mayor que)
`>=` (mayor que o igual que)

Operaciones lógicas (booleanas)

Resultados de estas operaciones son bool: True ó False

Resultados de estas operaciones son bool: True ó False

Condicionales: sintaxis

if

```
if (condición):  
    hacer esto
```

elif

```
if (condición):  
    hacer esto  
elif (condición):  
    hacer esto otro
```

else

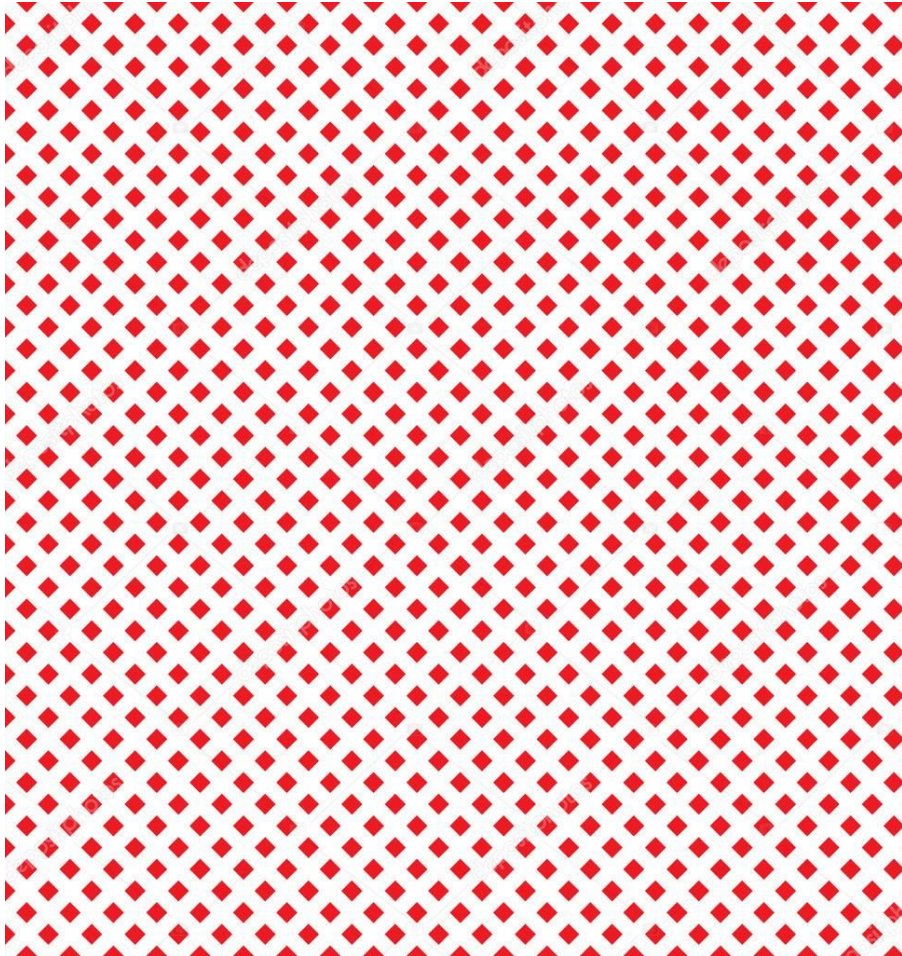
```
if (condición):  
    hacer esto  
else:  
    hacer esto otro
```

CICLOS

for

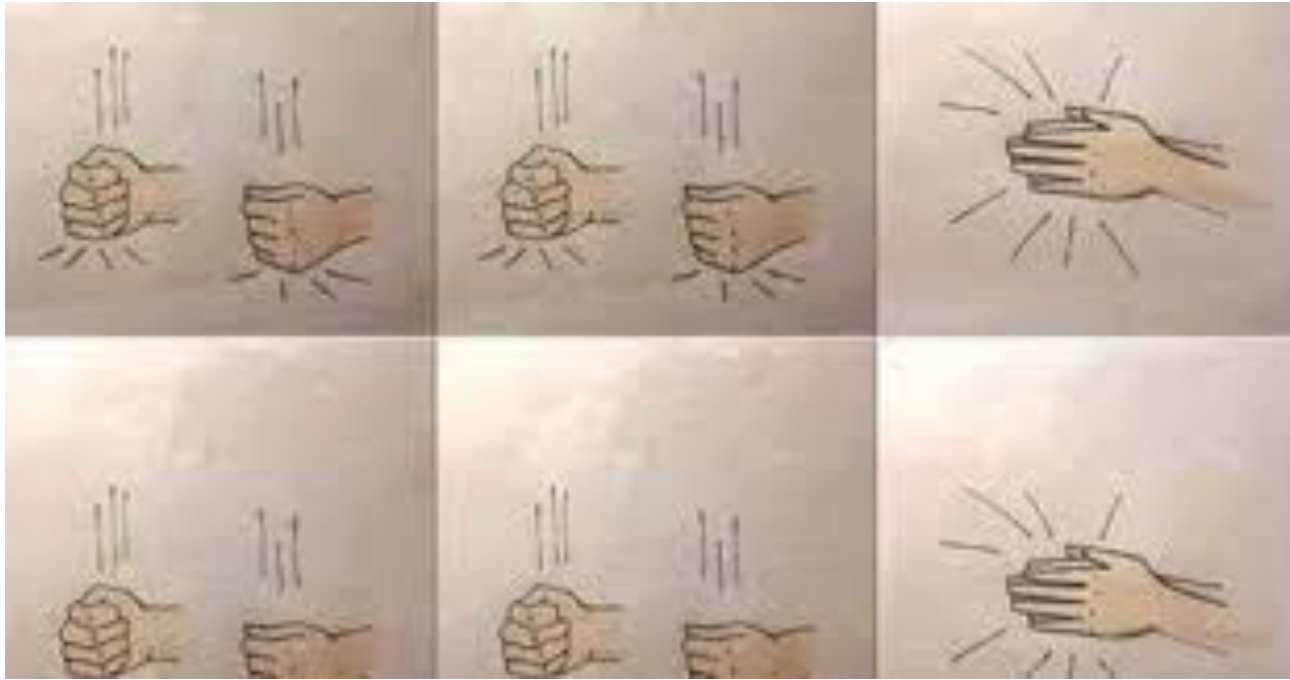
while

Repetición en el arte y diseño...



Patrones.

Repetición en el arte y diseño...



Patrones.

Ciclo for

variable de control,
iterador

elemento iterable

```
for variable in (lista, cadena, range, etc):  
    cuerpo del ciclo
```

```
//código fuera del ciclo
```

Ciclos (loops): while

variable de control,
iterador

```
i = 0
```

```
while i < 100:
```

```
    i += 1    elemento iterado
```

```
    print(i)
```

Ciclos (loops): while

¿Y qué pasa en este caso?

```
i = 0  
while i > 1:  
    i += 1  
    print(i)
```

Esto no tiene término, y el programa se va a quedar pegado...
Ustedes no quieren eso...



LISTAS

Listas

El mundo está lleno de listas.



ABARZÚA	REYES	FERNANDO NICOLÁS
ACEVEDO	GÓMEZ	DANIELA PAZ
ALIAGA	PEREZ	CRISTIAN AUGUSTO
ANDRADE	LEÓN	MARÍA JOSÉ DE JESÚS
ARNAIZ	CASTILLO	BASTIAN XABIER
BARRIENTOS	BAEZA	AGUSTIN SALVADOR
BRAVO	PARRA	DANIEL ESTEBAN
CAMHI	GUERRERO	NICOLÁS DAVID
CARABALL	MIERI	JOSÉ THOMAS
CASASSUS	KUFEKE	NICOLÁS
CERDA	BROWNE	SEBASTIÁN ANDRÉS
CHADWICK	ALEMPARTE	VICENTE
CONTRERAS	MORGADO	ISMAEL
CORTÉS	BRAVO	DIEGO GONZALO
CORTÉS	DE LA FUENTE	TOMÁS IGNACIO
DAROCH	AHUMADA	JOSEFINA FRANCISCA
DAWABE	FAJARDO	JAVIERA ANDREA
DE GEYTER	MESSINA	MATÍAS
DÍAZ	GONZÁLEZ	FERNANDA ISIDORA
DOMEYKO	HURTADO	TOMÁS
DONGHI	PEÑA	ROMINA RHYLYAN
ECHEVERRÍA	SOTO	EDUARDO IGNACIO
FIGUEROA	ORTEGA	BORJA
FLORES	MELLA	TOMÁS FRANCISCO

Listas

Definición:

Una lista es una serie *mutable* e *indexable* de elementos.

Sintaxis

```
lista = [ elemento1, elemento2, elemento3, ... ]
```

Ejemplos

```
l1 = [] # lista vacía
```

```
l2 = [3,4,2,4,9,6] # lista de números
```

```
l3 = ["a","b","c","d"] # lista de strings
```

```
l4 = ["a",3.4,True] # lista mixta
```

```
l5 = [[1,2,3],[4,5],[6,7,8]] # lista de listas
```

Listas

Indexable: Sus elementos se obtienen indicando su índice.

```
l = [3,4,2,4,9,6]    # lista de números
```

```
print(l[3])    # >>> 4
```

```
print(l[-2])    # >>> 9
```

Mutable: Sus elementos pueden ser modificados.

```
l = [3,4,2,4,9,6]    # lista de números l[0] = 6
```

```
print(l)    # >>> [6,4,2,4,9,6]
```

Listas, cómo recorrerlas

for: permite recorrer una lista (o string) y ejecutar código para cada elemento en ella.

Sintaxis

```
for     variable     in     lista:  
    bloque_de_codigo_for  
  
    ...  
    bloque_de_codigo_for
```

```
l = ['fruta', 'carne', 'arroz', 'café']
```

```
for e in l: # Recorre cada elemento de "l"  
    print("No olvides comprar", e) # Operamos con "e" actual
```

Listas, qué se puede hacer con ellas...

- Tamaño
- Acceder a una posición
- Iterar de forma determinada toda una lista
- Recorrer/buscar en una lista
- Pertenece a una lista
- Particionar/recomponer una lista
- Modificar una lista
- Métodos

OTRAS REFERENCIAS

Python Visually

<http://www.pythonvisually.com/>

Python for Designers

<https://pythonfordesigners.com/>

Tipografía para pantalla

https://issuu.com/ricardov_net/docs/tipografia_y_codigo

Libros Interactive Python (Ch: Simple Python Data)

<http://interactivepython.org/courselib/static/thinkcspy/index.html>

Think Python (Ch 2)

<http://greenteapress.com/wp/think-python/>

Pensar Python (Ch 2) <http://www.cmat.edu.uy/cmat/cursos/licenciatura/cursos/computacion/documentos/Think%20Python-esp.pdf/view>

Variables and analogies

<http://cs-blog.khanacademy.org/2013/09/teaching-variables-analogies-and.html>

Variables

<http://cscircles.cemc.uwaterloo.ca/1-variables/>